

User Input y MÁS movimiento 2D

Input por Teclado

¿Cómo?

- En cada frame vamos a querer saber si el usuario está presionando alguna tecla

¿Cómo?

- En cada frame vamos a querer saber si el usuario está presionando alguna tecla



Preguntamos en el `update()` si se está presionando alguna tecla

Update()

```
def update(dt):  
  
    if controller.is_key_pressed("Alguna tecla"):  
        #Que hacemos cuando la tecla se presiona
```

Ejemplo: Movimiento en el eje X

```
def update(dt):  
    #movimiento hacia la izquierda  
    if controller.is_key_pressed(pyglet.window.key.A):  
        personaje.position[0] -= dt  
  
    #movimiento hacia la derecha  
    if controller.is_key_pressed(pyglet.window.key.D):  
        personaje.position[0] += dt
```

Variables por tecla

Pueden revisar aquí la documentación de Pyglet -> Documentación
Donde están los nombres de todas las teclas.

Por ejemplo:

```
pyglet.window.key.W # W  
pyglet.window.key.S # S  
pyglet.window.key._1 # 1  
pyglet.window.key._2 # 2  
pyglet.window.key.RETURN # Enter  
...  
etc
```

Input por mouse

¿Cómo?

Para obtener informacion de lo que ocurre con el mouse debemos definir alguna de las siguientes funciones en nuestro código:

- `on_mouse_motion(x, y, dx, dy)`
 - a) **x** e **y** representan la posición del mouse (relativo a la esquina inferior izquierda (0,0))
 - b) **dx** y **dy** es la diferencia entre la posicion anterior y la presente

Más funciones...

- `on_mouse_press(x, y, button, modifiers)`
- `on_mouse_release(x, y, button, modifiers)`
- `on_mouse_drag(x, y, dx, dy, buttons, modifiers)`

En el parámetro **buttons** queda guardado cual botón fue presionando.

Va ser una de las siguientes opciones:

- `pyglet.window.mouse.LEFT`
- `pyglet.window.mouse.MIDDLE`
- `pyglet.window.mouse.RIGHT`

Ojo, **buttons** es una combinación bitwise de todos los botones que están siendo apretados...

Ojo, **buttons** es una combinación bitwise de todos los botones que estan siendo apretados...

Paréntesis para explicar BITS



Para más informacion → Bitwise Operation

Uso

#definimos la función

```
def on_mouse_drag(x, y, dx, dy, buttons, modifiers):  
    # vemos si comparten bits  
    if buttons & mouse.LEFT:  
        # ejecutamos algo en el click  
        abrirMenu()
```

Más funciones...

- `on_mouse_enter(x, y)` → cuando el mouse entra en la ventana
- `on_mouse_leave(x, y)` → cuando el mouse sale de la ventana
- `on_mouse_scroll(x, y, scroll_x, scroll_y)` → cuando se scrolea
 - Con scroll, por lo general solo se usa `scroll_y`, raramente salvo para los mouse raros se usa `scroll_x`

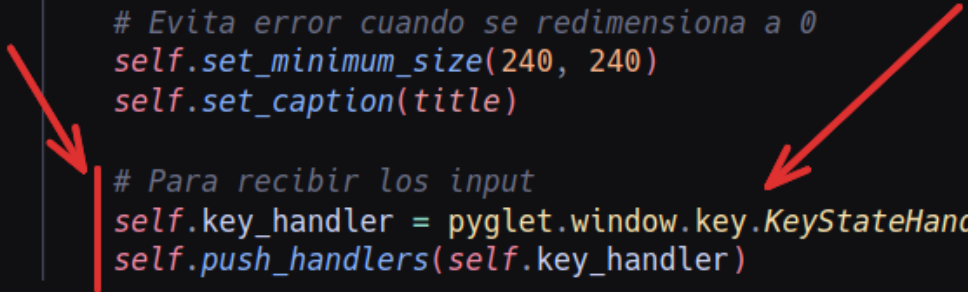
Para mas detalle → Documentación

Configuración extra

Para acceder a estas funcionalidades debemos agregarle a nuestro controlador las siguientes líneas:

- en el `__init__`

```
class Controller(pyglet.window.Window):  
    def __init__(self, title, *args, **kwargs):  
        super().__init__(*args, **kwargs)  
  
        # Evita error cuando se redimensiona a 0  
        self.set_minimum_size(240, 240)  
        self.set_caption(title)  
  
        # Para recibir los input  
        self.key_handler = pyglet.window.key.KeyStateHandler()  
        self.push_handlers(self.key_handler)
```



Configuración extra

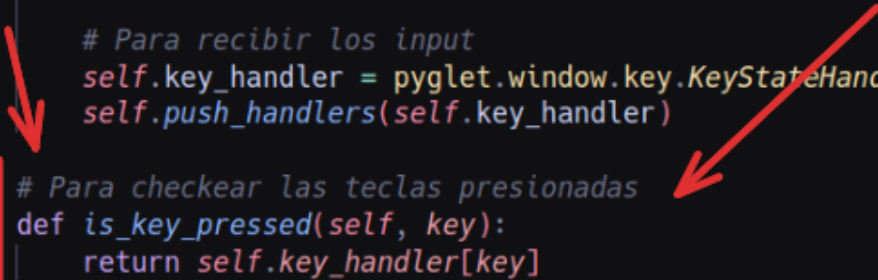
- Y fuera del `__init__` pero aun en el controlador

```
class Controller(pyglet.window.Window):
    def __init__(self, title, *args, **kwargs):
        super().__init__(*args, **kwargs)

        # Evita error cuando se redimensiona a 0
        self.set_minimum_size(240, 240)
        self.set_caption(title)

        # Para recibir los input
        self.key_handler = pyglet.window.key.KeyStateHandler()
        self.push_handlers(self.key_handler)

        # Para checkear las teclas presionadas
        def is_key_pressed(self, key):
            return self.key_handler[key]
```



Ejemplo

